Deep Learning Notes

Weeks 9–12: CNNs, Word Embeddings, RNNs & Transformers

IITM BS – Exam Preparation

Deon Dmello

August 2026

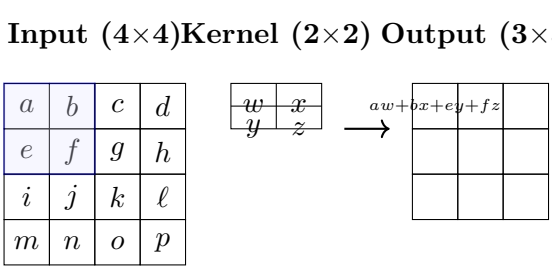
Contents

1 Week 9: Convolutional Neural Networks (CNNs)

1.1 Convolution Operation

The 2D convolution (technically cross-correlation) of input I with kernel K:

$$g(i,j) = \sum_{a=0}^{m-1} \sum_{b=0}^{n-1} I(i+a, j+b) \cdot K(a,b)$$



1.2 Output Size Formula

CNN Output Size – MEMORIZE THIS

$$\boxed{W_{\text{out}} = 1 + \frac{W_{\text{in}} - F + 2P}{S}}$$

where W_{in} = input size, F = filter size, P = padding, S = stride.

Same padding (output = input when $S = 1$): $P = \dfrac{F-1}{2}$

Number of output channels = Number of filters K

1.3 Parameter Counting in Conv Layers

Conv Layer Parameters

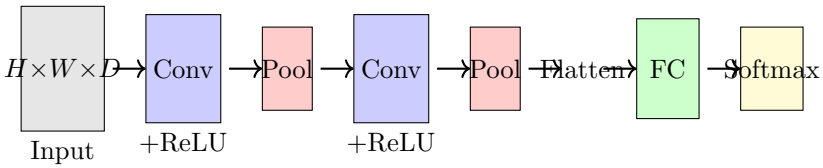
With bias: $F \times F \times D_{\text{in}} \times K + K$

Without bias: $F \times F \times D_{\text{in}} \times K$

where F = filter spatial size, D_{in} = input depth/channels, K = number of filters.

Pooling layers have ZERO learnable parameters!

1.4 CNN Architecture Diagram



1.5 Pooling

- **Max Pooling**: Takes max value in each window region
- **Average Pooling**: Takes average value in each window region
- Same output size formula applies (using pool size as F, pool stride as S)
- **No learnable parameters** in pooling layers

1.6 Backpropagation in CNNs

- **Max Pool**: gradient flows only to the position of the max value
- **Avg Pool**: gradient distributed equally to all positions
- **Conv layer weight gradient**: convolve input X with upstream gradient
- **Conv layer input gradient**: convolve weights W (flipped) with upstream gradient

1.7 Famous Architectures (Key Facts)

Network	Key Idea	Params
AlexNet	227×227×3 input, ReLU, dropout	∼27M
VGGNet	All 3×3 kernels, deeper	∼138M
GoogLeNet	Inception module (multi-size filters)	∼5M
ResNet	Skip connections: $f(x) = x + \mathcal{F}(x)$	varies

ResNet Skip Connection

$f(x) = x + \text{ReLU}(W \cdot x)$
If the layer learns nothing useful, $W \approx 0$, so $f(x) \approx x$ (identity). The network can *at least* preserve information. This solves vanishing gradients in deep networks.

1.8 Worked Example: Full CNN Computation

Example: Conv + Pool Pipeline

Input: 5×5 grayscale image. **Architecture**: Conv(3×3, stride 1, no padding, 1 filter, no bias) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2, stride 2).
Step 1: Output after Conv $= 1 + \frac{5-3+0}{1} = 3$. Output: 3×3
Step 2: After MaxPool $= 1 + \frac{3-2}{2} = 1$. Output: 1×1 (scalar)
Parameters: Only the 3×3 kernel = **9 parameters**. No bias. Pool has 0.

2 Week 10: Word Representations & Embeddings

2.1 The Problem with One-Hot Vectors

For vocabulary $|V|$, each word is a vector of size $|V|$ with a single 1.

- Euclidean distance between any two words: $\sqrt{2}$
- Cosine similarity between any two words: 0
- No notion of semantic similarity captured
- Very sparse and high-dimensional

2.2 Co-occurrence Matrix

Corpus: "cat sat mat", "dog sat mat" (window = 1)

	cat	sat	mat	dog
cat	0	1	0	0
sat	1	0	2	1
mat	0	2	0	0

2.3 PMI (Pointwise Mutual Information)

PMI Formula

$$\text{PMI}(w, c) = \ln\left(\frac{\text{count}(w, c) \times N}{\text{count}(w) \times \text{count}(c)}\right)$$

where N = total number of (word, context) pairs in the matrix.
PPMI $= \max(0, \text{PMI})$ (set negative values to 0)

2.4 SVD for Dimensionality Reduction

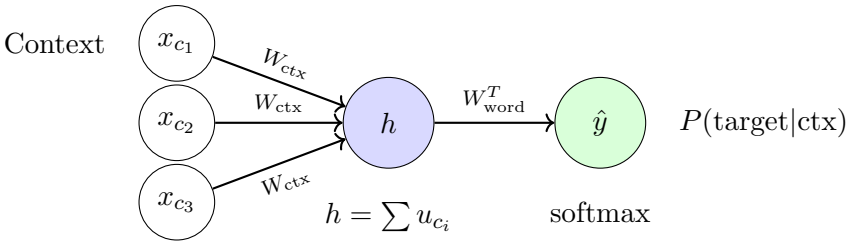
Given PPMI matrix $X_{m\times n}$, SVD gives rank-k approximation:

$$\hat{X} = U_{m\times k}\Sigma_{k\times k}V^T_{k\times n}$$

Word representation: $W_{\text{word}} = U\Sigma \in \mathbb{R}^{m\times k}$

Benefits of SVD: Reduces sparsity, produces dense low-dimensional embeddings, captures latent semantic similarity.

2.5 CBOW (Continuous Bag of Words)



CBOW Forward Pass – EXAM CRITICAL

Step 1: Select context word columns from W_{context}:

$$h = \sum_{\text{context words}} u_{c_i} \quad \text{(sum of columns from } W_{\text{context}}\text{)}$$

Step 2: Compute scores for each word:

$$z_j = v_j^T \cdot h \quad \text{where } v_j \text{ is column } j \text{ of } W_{\text{word}}$$

Step 3: Apply softmax:

$$P(w = j|\text{context}) = \hat{y}_j = \frac{e^{z_j}}{\sum_{w' \in V} e^{z_{w'}}}$$

Loss: $L = -\ln(\hat{y}_{\text{true}})$

2.6 Skip-gram Model

Opposite of CBOW: given a **target word**, predict **context words**.

Skip-gram Pair Counting – EXAM FAVORITE

For sentence of length L, window size d (each side):

- Interior words (not near edges): contribute $2d$ pairs each
- Edge words: contribute fewer pairs
- Total $= \sum_{i=0}^{L-1} \left[\min(d, i) + \min(d, L-1-i)\right]$

Example: "deep learning helps build models", $d = 2$:

deep (0)	$\rightarrow$	min(2,0)+min(2,4) = 0+2 = 2
learning (1)	$\rightarrow$	min(2,1)+min(2,3) = 1+2 = 3
helps (2)	$\rightarrow$	min(2,2)+min(2,2) = 2+2 = 4
build (3)	$\rightarrow$	min(2,3)+min(2,1) = 2+1 = 3
models (4)	$\rightarrow$	min(2,4)+min(2,0) = 2+0 = 2

Total = 2+3+4+3+2 = **14**

2.7 Negative Sampling & Hierarchical Softmax

Negative Sampling: Instead of full softmax over $|V|$ words, sample k negative examples. Maximize:

$$\sum_{(w,c) \in D} \ln \sigma(v_c^T v_w) + \sum_{(w,r) \in D'} \ln \sigma(-v_r^T v_w)$$

Hierarchical Softmax: Build binary tree with $|V|$ leaves. Probability computed by traversing path from root to leaf node. Cost: $O(\log |V|)$ instead of $O(|V|)$.

2.8 GloVe

Combines count-based and prediction-based:

$$\min_{v_i, v_j, b_i, b_j} \sum_{i,j} f(X_{ij}) \left(v_i^T v_j + b_i + b_j - \ln X_{ij}\right)^2$$

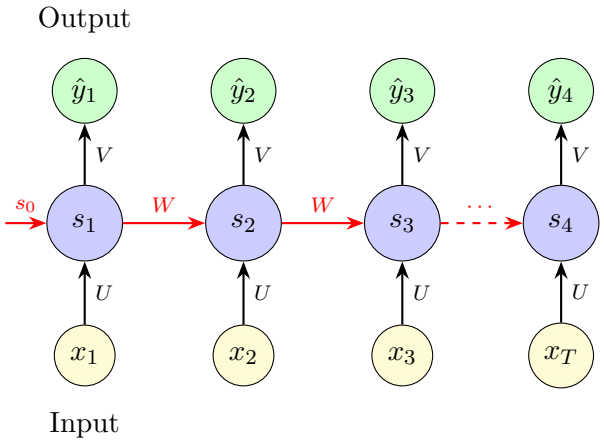
3 Week 11: Recurrent Neural Networks & LSTMs

3.1 Why Sequences?

Fixed-size feedforward networks cannot handle:

- Variable-length inputs (sentences, time series)
- Dependencies between positions ("The cat sat on the mat")
- Tasks where order matters

3.2 RNN Architecture



RNN Equations

$$s_t = \sigma(U \cdot x_t + W \cdot s_{t-1} + b) \qquad \hat{y}_t = O(V \cdot s_t + c)$$

Parameters: $U \in \mathbb{R}^{n \times d}$, $W \in \mathbb{R}^{d \times d}$, $V \in \mathbb{R}^{d \times k}$, $b \in \mathbb{R}^d$, $c \in \mathbb{R}^k$
Total params (with bias): $nd + d^2 + dk + d + k$

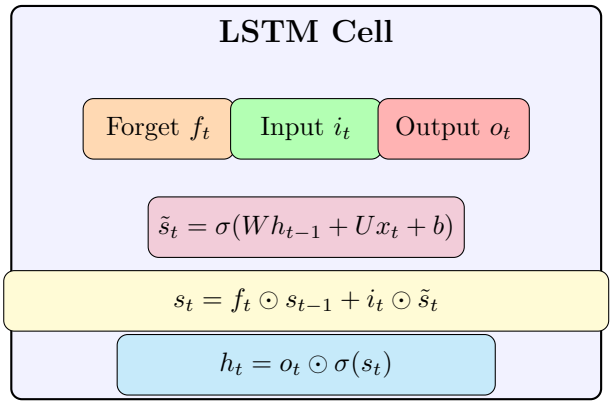
Key: Parameters U, W, V are **shared across all time steps**!
Sequence length T does NOT affect parameter count.

3.3 Backpropagation Through Time (BPTT)

- Total loss $= \sum_t L_t$ (sum over all time steps)
- Gradient of W involves chain of products through time $\rightarrow$ vanishing/exploding gradients
- Truncated BPTT: limit backprop to τ steps

3.4 LSTM (Long Short-Term Memory)

$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$



LSTM Equations – MUST KNOW

Three gates:

$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f) \quad \text{(forget gate)}$$
$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i) \quad \text{(input gate)}$$
$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o) \quad \text{(output gate)}$$

State update:

$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b) \quad \text{(candidate state)}$$
$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t \quad \text{(cell state)}$$
$$h_t = o_t \odot \sigma(s_t) \quad \text{(hidden state / output)}$$

LSTM Parameter Count – HIGH VALUE

4 gate/state equations, each with: $W_x \in \mathbb{R}^{n \times d}$, $W_h \in \mathbb{R}^{d \times d}$, $b \in \mathbb{R}^d$
Per gate: $nd + d^2 + d$. Total for 4 gates: $4(nd + d^2 + d) = 4d(n + d + 1)$
Output layer: $V \in \mathbb{R}^{d \times k}$, $c \in \mathbb{R}^k \Rightarrow dk + k = k(d + 1)$

$$\boxed{\text{Total LSTM params} = 4d(n + d + 1) + k(d + 1)}$$

LSTM has $\sim$4$\times$ the parameters of a vanilla RNN!
Sequence length T does NOT affect the count.

3.5 GRU (Gated Recurrent Unit)

- 2 gates (update + reset) instead of 3
- No separate cell state; forget and input gates are *tied*: $(1 - i_t) \odot s_{t-1} + i_t \odot \tilde{s}_t$
- Fewer parameters than LSTM: $3d(n + d + 1) + k(d + 1)$

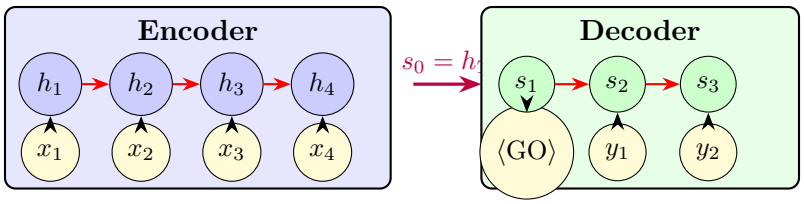
3.6 Worked Example: RNN Forward Pass

RNN Computation Example

$U = 1$, $W = 1$, $s_0 = 0$. Inputs: $x_1 = -\ln 3$, $x_2 = -0.25$.
Step 1: $s_1 = \sigma(U \cdot x_1 + W \cdot s_0) = \sigma(-\ln 3 + 0) = \frac{1}{1+e^{\ln 3}} = \frac{1}{1+3} = 0.25$
Step 2: $s_2 = \sigma(U \cdot x_2 + W \cdot s_1) = \sigma(-0.25 + 0.25) = \sigma(0) = 0.5$
Output: If $V = 2$, linear output: $\hat{y} = V \cdot s_2 = 2 \times 0.5 = 1.0$

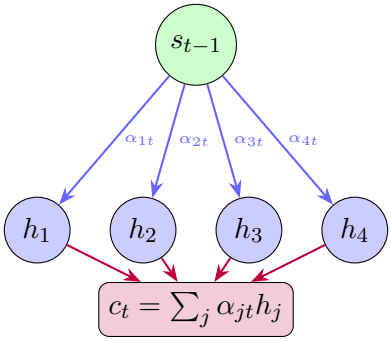
4 Week 12: Attention Mechanism & Transformers

4.1 Encoder-Decoder Model



4.2 Attention Mechanism

Instead of compressing the entire input into one vector h_T, attention lets the decoder **look back at all encoder hidden states** at each decoding step.



Attention Score Computation – EXAM CRITICAL

Step 1: Compute raw scores:

$$e_{jt} = V_{\text{att}}^T \tanh(U_{\text{att}} \cdot s_{t-1} + W_{\text{att}} \cdot h_j)$$

Step 2: Normalize with softmax:

$$\alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{j'} \exp(e_{j't})}$$

Step 3: Compute context vector:

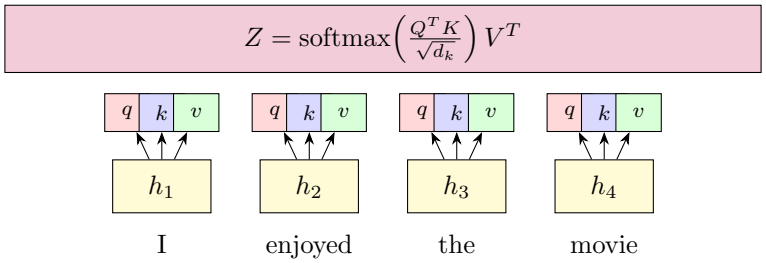
$$c_t = \sum_j \alpha_{jt} \cdot h_j$$

4.3 Self-Attention (Transformer Foundation)

In self-attention, each word attends to **all other words** in the same sequence.

For each input embedding h_j, create three vectors:

$$q_j = W_Q h_j \quad \text{(query)}, \qquad k_j = W_K h_j \quad \text{(key)}, \qquad v_j = W_V h_j \quad \text{(value)}$$



Self-Attention Output

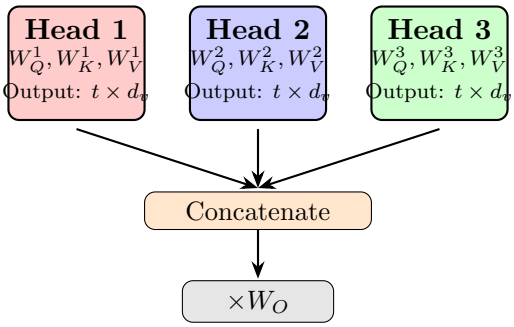
Stacking all queries, keys, values:

$$Q = W_Q H, \quad K = W_K H, \quad V = W_V H \qquad \text{where } H = [h_1, \ldots, h_T]$$

$$\boxed{Z = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V^T}$$

Scaling by $\sqrt{d_k}$ prevents dot products from becoming too large.

4.4 Multi-Head Attention



Each head has its own W_Q^i, W_K^i, W_V^i. Outputs are concatenated and projected through W_O.

Multi-Head Attention Parameters – EXAM FAVORITE

Given: d_{model}, h heads, $d_k = d_q = d_{\text{model}}/h$, d_v.
Per head: $W_Q(d_k \times d_{\text{model}}) + W_K(d_k \times d_{\text{model}}) + W_V(d_v \times d_{\text{model}})$
All h heads: $h \times (d_k + d_k + d_v) \times d_{\text{model}}$
Output projection: $W_O \in \mathbb{R}^{d_{\text{model}} \times (h \cdot d_v)}$

$$\boxed{\text{Total MHA (no bias)} = h(2d_k + d_v) \cdot d_{\text{model}} + d_{\text{model}} \cdot (h \cdot d_v)}$$

4.5 Attention Output Size

Attention Output Shape

$Q \in \mathbb{R}^{d_k \times t}$, $K \in \mathbb{R}^{d_k \times t} \Rightarrow Q^T K \in \mathbb{R}^{t \times t}$
After softmax: $t \times t$. Multiply by $V^T \in \mathbb{R}^{t \times d_v}$:

$$\text{Output per head} = t \times d_v \quad \Rightarrow \quad \text{elements} = t \cdot d_v$$

4.6 Masked Self-Attention

Used in the **decoder** to prevent looking at future tokens.

- Mask matrix M: diagonal $= 0$, upper triangle $= -\infty$
- $\text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}} + M\right) V^T$
- Sum of diagonal elements of $M = 0$ (token can see itself)
- Total attention scores $= B \times h \times T \times T$

4.7 Feed-Forward Network (FFN)

Applied position-wise after attention:

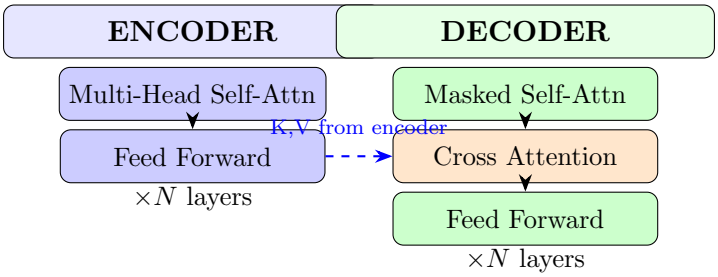
$$\text{FFN}(z) = \max(0, z \cdot W_1 + b_1) \cdot W_2 + b_2$$

FFN Parameters

$W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{ff}}$, $b_1 \in \mathbb{R}^{d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_{\text{model}}}$, $b_2 \in \mathbb{R}^{d_{\text{model}}}$

$$\boxed{\text{FFN params} = 2 \cdot d_{\text{model}} \cdot d_{ff} + d_{ff} + d_{\text{model}}}$$

4.8 Transformer Architecture Summary



Component	Description
Encoder Self-Attention (P)	Each source token attends to all source tokens
Masked Self-Attention (Q)	Prevents access to future target tokens
Cross Attention (R)	Queries from decoder, K/V from encoder
FFN (S)	Applied position-wise after attention

4.9 Worked Example: Transformer Parameter Counting

Full Transformer Param Count

$t = 32$, $d_{\text{model}} = 64$, $h = 2$, $d_v = 16$, $d_{ff} = 256$.
MHA: $d_k = d_q = 64/2 = 32$
Per head: $W_Q(32 \times 64) + W_K(32 \times 64) + W_V(16 \times 64) = 2048 + 2048 + 1024 = 5120$
2 heads: $5120 \times 2 = 10240$
W_O: $64 \times (2 \times 16) = 64 \times 32 = 2048$
Total MHA $= 10240 + 2048 =$ **12288**

FFN (with bias): $64 \times 256 + 256 + 256 \times 64 + 64 = 16384 + 256 + 16384 + 64 =$ **33088**

5 Quick Reference: Numerical Tricks

Sigmoid values:

$$\begin{aligned}\sigma(0) &= 0.5\\ \sigma(1) &\approx 0.731\\ \sigma(-1) &\approx 0.269\\ \sigma(2) &\approx 0.881\\ \sigma(-\ln 3) &= 0.25\end{aligned}$$

tanh values:

$$\begin{aligned}\tanh(0) &= 0\\ \tanh(1) &\approx 0.762\\ \tanh(2) &\approx 0.964\\ \tanh(3) &\approx 0.995\end{aligned}$$

Exponentials:

$$\begin{aligned}e^0 = 1, \quad e^1 &\approx 2.718\\ e^2 \approx 7.389, \quad e^{-1} &\approx 0.368\\ \ln 2 &\approx 0.693\\ \ln 3 &\approx 1.099\end{aligned}$$

Key derivative shortcuts:

$$\begin{aligned}\sigma'(x) &= \sigma(x)(1-\sigma(x))\\ \tanh'(x) &= 1-\tanh^2(x)\\ \frac{\partial L}{\partial a_L} &= \hat{y} - y \quad \text{(softmax+CE)}\end{aligned}$$